

Easy Realistic Horror and Survival Game Creator

1. Introduction

Horror and Survival Game Creator is a complete Unity toolkit for building immersive first-person horror games.

It provides ready-to-use scripts, prefabs, and systems for player control, enemy AI, inventory, interaction, jumpscare, and more.

2. Getting Started

1. Import the Asset

Download and import the package into your Unity project (recommended Unity 2022.3+).

2. Project Settings

- Ensure you have the new Input System enabled (Edit > Project Settings > Player > Active Input Handling > Both or New Input System).
- Check all the layers and tags imported correctly (e.g., "Player", "Enemy", "Item", "Demon", etc.).
- High Definition Render Pipeline (HDRP) Required:
 - This project is designed for Unity's HDRP.
 - Make sure HDRP is installed via Window > Package Manager.
 - After importing the asset, go to Edit > Project Settings > Graphics and assign the HDRP Asset.
 - If you see pink/missing materials, reimport the HDRP package and assign the correct HDRP asset.

3. Project Structure

• Animations

This folder contains all animation assets used in the project, including character movements, hand actions, enemy behaviors, and environmental effects. You can find Animator Controllers and Animation Clips here, which are referenced by prefabs and scripts to bring dynamic motion to your game.

• Documentation

Here you will find guides, manuals, and reference documents that explain how to use the asset, set up your project, and customize features. Always check this folder for the latest instructions, update notes, and best practices.

• Materials

All material assets for rendering surfaces in the game are stored here. These materials define the visual appearance of models, such as metal, wood, skin, and glass, and are optimized for HDRP. You can modify or create new materials to match your game's style.

• Models

This folder contains 3D models used throughout the project, including characters, props, environment pieces, and interactive objects. Models are typically in FBX or OBJ format and are ready to be used in prefabs or directly in your scenes.

- **Prefabs**

Prefabs are reusable game objects with pre-configured components and settings. In this folder, you will find player, enemy, item, UI, and environment prefabs. Drag and drop these into your scenes to quickly build gameplay elements without manual setup.

- **Scenes**

All Unity scene files are located here. Each scene represents a different level, menu, or environment in your game. You can use the provided scenes as templates or create new ones to expand your project.

- **Settings**

Project and asset settings files are stored here, including HDRP configuration, input settings, and custom asset settings. Make sure these are correctly set up for optimal performance and compatibility.

- **Sounds**

All audio assets, such as sound effects, ambient noises, and music tracks, are found in this folder. These sounds are used to enhance the horror atmosphere and provide feedback for player actions and events.

- **Textures**

This folder contains image files used for materials, UI elements, and environmental details. Textures are optimized for HDRP and can be edited or replaced to fit your visual theme.

- **UIElements**

UIElements includes all user interface assets, such as buttons, panels, icons, and fonts. These are used in menus, HUDs, and in-game notifications. You can customize UIElements to create a unique user experience for your game.

4. Main Systems & Scripts

Player System

- HeroPlayerScript: Central player logic (health, interaction, hiding, inventory).
- FirstPersonController: Handles movement, jumping, crouching, sprinting, stamina.
- CameraLook: Mouse look and camera shake effects.

Enemy AI

- DemonScript: Enemy behavior, patrolling, chasing, attacking, and reacting to player actions.
- NavMeshAgent: Used for navigation and pathfinding.

Interaction System

- ItemScript: Base for all interactable items (doors, keys, notes, weapons).
- BoxScript, LadderScript, DrawerScript, etc.: Specialized item scripts.

UI & Game Management

- GameCanvas: Manages all UI panels, health, stamina, notes, hints, and game state.
- SettingsScript: Quality, difficulty, sound, and mouse sensitivity settings.

- MainMenuCanvas: Main menu logic and scene loading.

Audio

- AudioManager: Centralized sound effect and music playback.

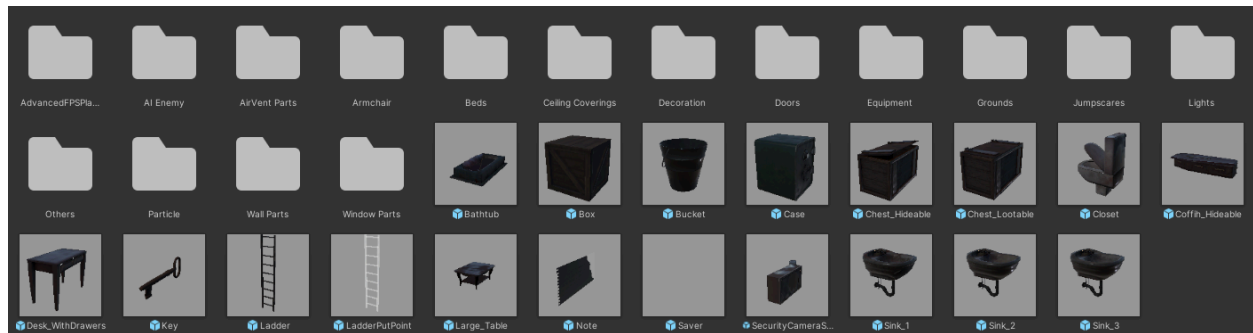
Saving System

- SaverScript: Checkpoints and player progress saving.

Jumpscare System

- JumpscareScript: Triggers for animations, object falls, and explosions to scare the player.

5. Ready to Use Prefabs & Their Usage



AdvancedFPSPlayer: Contains prefabs for the main player character, including camera, hands, and interaction points.

AI Enemy: Prefabs for enemy characters with built-in AI behaviors (e.g., Demon, Monster).

AirVent Parts: Prefabs for air vent systems, used for stealth and navigation.

Armchair: Prefabs for armchairs, often used as interactive furniture or hiding spots.

Beds: Bed prefabs, can be used for hiding or environmental decoration.

Ceiling Coverings: Ceiling-related prefabs for level design and environment building.

Decoration: Various decorative objects to enhance the atmosphere.

Doors: Prefabs for doors, including interactive and locked variants.

Equipment: Player equipment prefabs (e.g., flashlight, knife, cross, anomaly detector).

Grounds: Ground and floor prefabs for environment construction.

Jumpscare: Prefabs for jumpscare triggers and effects.

Lights: Light source prefabs, including electric lights and candles.

Others: Miscellaneous prefabs not covered by other categories.

Particle: Particle effect prefabs for visual feedback (e.g., smoke, sparks).

Wall Parts: Wall segment prefabs for building rooms and corridors.

Window Parts: Window prefabs for environment design.

Individual Prefabs

Bathtub: An interactable bathtub, can be used for hiding or as a puzzle element.

Box: Movable box, used for physics puzzles or blocking paths.

Bucket: Environmental prop, may be used for decoration or minor interactions.

Case: A case or chest, often used for storing loot or keys.

Chest_Hideable: Chest that the player can hide inside to avoid enemies.

Chest_Lootable: Chest that can be opened to collect items or resources.

Closet: Hideable closet, allows the player to evade enemies.

Coffin_Hideable: Coffin that serves as a hiding spot for the player.

Desk_WithDrawers: Desk with drawers, may contain loot or clues.

Key: Collectible key, used to unlock doors or progress in the game.

Ladder: Movable ladder, used for reaching higher areas or solving puzzles.

LadderPutPoint: Designated spot for placing ladders, often part of environmental puzzles.

Large_Table: Large table, used for decoration or as an obstacle.

Note: Collectible note, provides story, hints, or lore to the player.

Saver: Checkpoint prefab, saves player progress when reached.

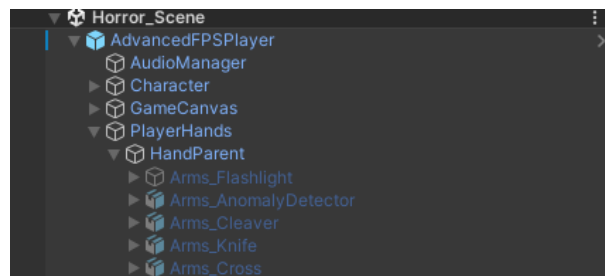
SecurityCameraScript: Security camera, may be used for surveillance or puzzles.

Sink_1 / Sink_2 / Sink_3: Different sink models, used for environmental detail or minor interactions.

Usage Tips:

1. Drag and drop these prefabs into your scene to quickly build environments, puzzles, and gameplay mechanics.
2. Interactive prefabs (e.g., Box, Key, Chest_Hideable)'re usually linked to scripts for behaviors.
3. Hideable prefabs (Closet, Coffin_Hideable, Chest_Hideable) are essential for gameplay.
4. Saver prefabs are important for checkpoint and save system functionality.

Hands:



All Hands are separated and for each equipment, we have an hand attached to Character object's Hero Controller script. If you want to add more hands and more equipment, you need to duplicate these hands, position your equipment into that new hand, create a new function and new variables for the hand and assign this new hand object to the new variable on inspector.



If you want to give an anomaly attribute to any object, you just need to add AnomalyScript component to that specific object. After that, the Detector will start to detect that object as an anomaly:



6. Customization & Extending

- Add New Items: Inherit from ItemScript and implement custom logic.
- Create New Enemies: Duplicate and modify DemonScript or create new AI scripts.
- UI Customization: Edit GameCanvas prefab and scripts for custom panels and feedback.
- Audio/Visuals: Replace or add new sounds, models, and animations for your theme.

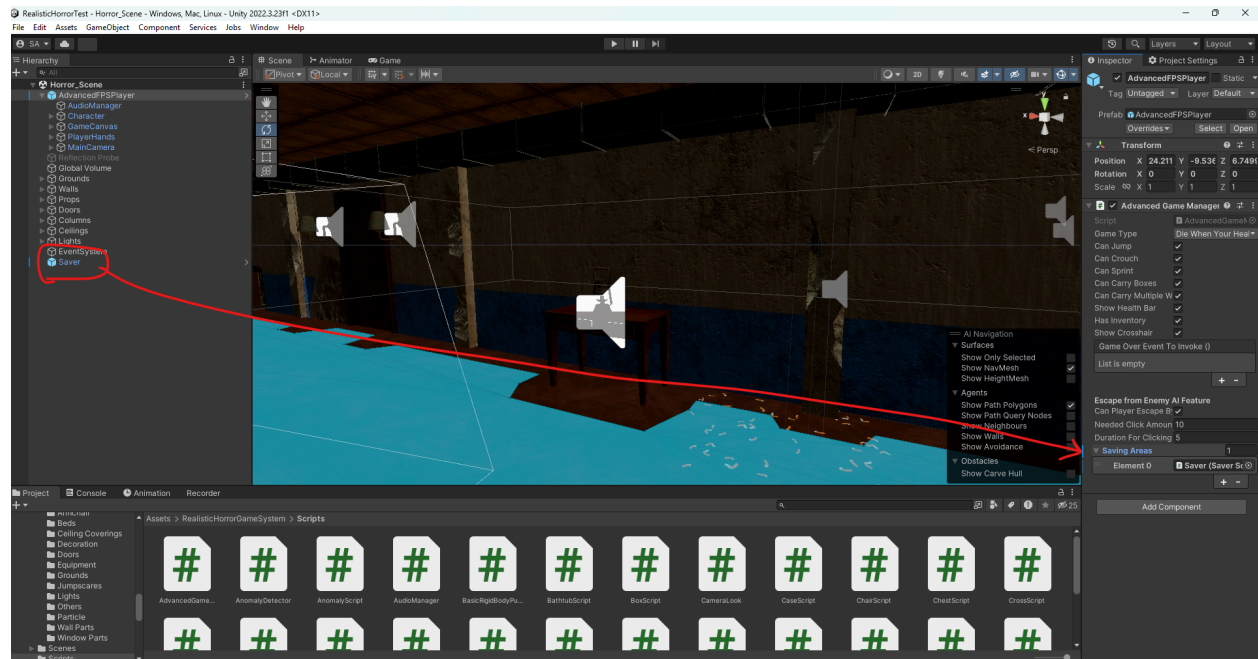
7. Input System

- Uses Unity's New Input System for all player actions.
- Keyboard, mouse, and gamepad supported.
- Input actions are defined in scripts (e.g., movement, jump, crouch, interact, attack).
- To change key bindings, edit the relevant InputAction in the script or use Unity's Input System asset.

8. Saving & Loading

- SaverScript: Place in scene as checkpoints.
- Player progress is saved using PlayerPrefs (SavingIndex).
- On game start, player is placed at last checkpoint.

When you add Saver objects to your scene, please don't forget to give them SavingIndex. (These indexes must be unique). At the end, please add these saver objects to AdvancedFPSPlayer's SavingAreas list on inspector:



9. Tips & Best Practices

- Layer & Tag Setup: Ensure all objects use correct layers/tags for interaction and AI.
- NavMesh: Bake your scene's NavMesh for enemy navigation.
- Prefabs: Always use provided prefabs for core systems to avoid setup errors.
- Testing: Playtest frequently and check console for errors or warnings.
- Extending: Use inheritance and UnityEvents for custom gameplay features.

10. Troubleshooting

- Input Not Working: Check Input System settings and ensure actions are enabled.
- UI Issues: Verify GameCanvas is present and all references are assigned.
- Enemy Navigation Problems: Make sure NavMesh is baked and agents are on correct layers.
- Saving Not Working: Confirm SaverScript is placed and SavingIndex is unique per checkpoint.

11. Contact & Support

For asset support, feature requests, or bug reports, contact us:) We are always here to help:
queendevolver95@gmail.com